Conclusion

Conclusion I

This paper presented a small, tested domain language for specifying controlled methods, informed by BPL's [@bpl2026] domain-language design for laboratory protocols and generalized to any controlled procedure. It validates a simple proposition: a controlled vocabulary expressed as typed, validated dataclasses — not a parsed text grammar — is sufficient to reproduce BPL's core safety properties at a scope appropriate for a template exemplar.

Exemplar achievements I

Operating as the methods-paper exemplar for the Research Project Template methodology, the project deployed the three foundational pillars:

1. **`src/methods_dsl/` library**: a controlled vocabulary, a dimensional unit system, four staged validation gates, a deterministic compiler, and four export formats — with no plotting, no file I/O, and (with one declared logging exception) no `infrastructure` imports.
2. **`tests/` integrity**: a zero-mock suite over real `Method` fixtures covering the success path and every gate-failure mode, under a 90% project coverage gate.
3. **`docs/` knowledge operations**: the correspondence with BPL's pipeline, testing philosophy, and operational rules that keep the library, scripts, and manuscript aligned.

Technical contributions I

Controlled vocabulary in code, not a parser

The hallmark of this exemplar is the design choice it demonstrates: a `.bpl` file's text grammar buys generality this template does not need, while a frozen-dataclass model buys construction-time validation (`__post_init__`) that a parsed AST would have to re-derive. The controlled vocabulary's discipline lives in the *types*, not in concrete syntax.

Honest handling of trust boundaries

`trust.py`'s hash-chain documents its own limit explicitly: it detects in-chain tampering but cannot detect a chain rewritten from record zero. This makes the provenance guarantee a visible, testable property rather than an implied cryptographic guarantee the implementation does not actually provide.

Key insights I

1. **Determinism follows from explicit tie-breaking**: Kahn's algorithm [@kahn1962topological] alone does not guarantee a stable schedule across runs — the ascending-`step_id` tie-break is what makes `plan_hash` reproducible, and [@sec:results] checks this live rather than asserting it.
2. **Staged short-circuit avoids misleading errors**: running `plan_gate` and `target_gate` against a structurally invalid method would report noise, not signal — `run_all_gates` short-circuits after the first two gates for exactly this reason.
3. **A controlled vocabulary generalizes by restraint, not by expansion**: `SensorCalibrationSweep` reuses every `StepKind` and `Target` the wet-lab-flavored `PBSPreparation` example uses; nothing was added to support a second domain.

Future extensions I

This foundation could be extended to:

- **A real `.bpl`-style parser**: add `grammar//parser//transformer/` stages ahead of the existing `model.py`, reusing every downstream gate and the compiler unchanged.
- **More backends**: a `robot` execution target with capability-aware lowering, mirroring BPL's Biomek translation layer.
- **A capability registry**: per-target primitive support declarations, generalizing BPL's `capabilities/` registry and `bplc capabilities` report.

Final assessment I

The `template_methods_paper` tree is the canonical reference for how a methods paper — a manuscript whose subject is a methodology — stays synchronized with the code implementing that methodology across rebuilds. The pipeline compiled both worked example methods, wrote `output/data/compiled_plans.json`, `output/reports/gate_report.json`, and `output/reports/trust_chain_report.json`, and rendered this markdown together with `config.yaml` into PDF.